

Algorithm

Question: 01

01. Express the time complexity of the following function in asymptotic notation.

Tweety (n)

```
{
    j = 1
    while j <= n*n
        j = j * (X MOD 3 + 2)
}
```

How the time complexity will be changed if the **multiplication (*)** is replaced by an **addition(+)**.

Ans:

- **Initial condition:** $j = 1$, and in each iteration, j is multiplied by $(X \text{ MOD } 3 + 2)$.
- After k iterations, j will be $1 * (X \text{ MOD } 3 + 2)^k$.
- The loop stops when $j > n^2$. So, solve for k :

$$(X \text{ MOD } 3 + 2)^k \geq n^2$$

- The time complexity is $O(\log n^2) = O(2 \log n)$. Since constants are ignored, the final complexity is $O(\log n)$.

If multiplication is replaced with addition:

- The update will be linear, so the time complexity will change to $O(n)$.

02. Express the time complexity of the following function in asymptotic notation.

void Wood (int n)

```
{
    for(int i = 1; i <= n; i = i*2)
        for(int j = 1; j <= i; j = j* (Y MOD 3+2))
            k = k + 1;
}
```

How the time complexity will be changed if the **multiplication (*)** is replaced by an **addition(+)**.

Ans:

Step 1: Outer Loop Analysis

The outer loop:

```
for (int i = 1; i <= n; i = i * 2)
```

- i starts at 1 and is multiplied by 2 in each iteration.
- The value of i becomes: 1, 2, 4, 8, 16, ..., up to n .
- Therefore, the number of iterations is approximately $\log_2(n)$.

Thus, the outer loop runs in **$O(\log n)$** time.

Step 2: Inner Loop Analysis

The inner loop:

```
for (int j = 1; j <= i; j = j * (Y MOD 3 + 2))
```

- j starts at 1 and is multiplied by a constant $(Y \text{ MOD } 3 + 2)$ in each iteration.
- Since $Y \text{ MOD } 3$ can be 0, 1, or 2, $(Y \text{ MOD } 3 + 2)$ can be 2, 3, or 4.
- Thus, j increases exponentially in each iteration.

Therefore, the inner loop runs approximately **$O(\log i)$** times.

At most, when $i \approx n$, the inner loop runs **$O(\log n)$** times.

Step 3: Overall Time Complexity

- Outer loop runs **$O(\log n)$** times.
- For each outer iteration, the inner loop runs **$O(\log n)$** times.

Thus, the total time complexity:

$$O(\log n \times \log n) = O((\log n)^2)$$

Final time complexity: **$O((\log n)^2)$** .

Step 4: If Multiplication is Replaced by Addition

If we replace $i = i * 2$ and $j = j * (Y \text{ MOD } 3 + 2)$ with addition:

- Outer loop becomes linear: **$O(n)$** .
- Inner loop becomes linear in i : **$O(i)$** .

Thus, the total time complexity becomes:

$$O(1+2+3+\dots+n) = O(n^2) \quad O(1 + 2 + 3 + \dots + n) = O(n^2)$$

New time complexity: **$O(n^2)$** .

03. What is the smallest value of n such that an algorithm whose running time is $70 \times (X+3) \times n^2$ runs after faster than an algorithm whose running time is 2^n on the same machine?

Ans:

Given:

- First algorithm: $70 \times (X + 3) \times n^2$
- Second algorithm: 2^n Find n such that

$$70 \times (X+3) \times n^2 < 2^n$$

Steps to solve:

1. Simplify the equation:

$$70 \times (X+3) \times n^2 < 2^n$$

2. For the smallest value of n , substitute values for X (since X is a constant value).

For simplicity, let $X = 0$ (to get the smallest value):

$$\begin{aligned} 70 \times 3 \times n^2 &< 2^n \\ 210 \times n^2 &< 2^n \end{aligned}$$

Now, you can solve this inequality numerically by checking different values of n to find when the inequality holds true.

For example:

- $N = 5$: $210 \times 5^2 = 5250$, and $2^5 = 32$, so this is false.
- $N = 10$: $210 \times 5^2 = 21000$, and $2^5 = 1024$, so this is false.
- $N = 15$: $210 \times 5^2 = 47250$, and $2^5 = 32768$, so this is false.
- $N = 20$: $210 \times 5^2 = 84000$, and $2^5 = 1048576$, so this is true.

Thus, the smallest n is **20**.

04. Find the time complexity of the following function?

Fib (n)

```
if ( $n = 0$  or  $n = 1$ ) return 1
return Fib( $n-1$ ) + Fib( $n-2$ )
```

How will it be changed if the function is written as follows?

Fib tail (n , $a=0$, $b=1$)

```
if ( $n == 0$ ) return a
```

```

    if (n == 0)    return b
    return Fib_tail(n-1, b, a+b)

```

Here, a=0, b=1 are default arguments.

Ans:

```

Fib (n)
    if (n = 0 or n = 1) return 1
    return Fib(n-1) + Fib(n-2)

```

The recurrence relation for this function is:

$$T(n) = T(n-1) + T(n-2) + O(1)$$

Time complexity: **$O(2^n)$**

```

Fib_tail(n, a=0, b=1) {
    if (n == 0) return a;
    if (n == 1) return b;
    return Fib_tail(n-1, b, a+b);
}

```

The recurrence relation for this function is:

$$T(n)=T(n-1) + O(1)$$

Time complexity: **$O(n)$**

05. Find the time complexity of the following function.

```

void Tweety (int n)
{
    for(int i = 1; i <= n; i = i++)
        for(int j = 1; j <= i*i; j++)
            k = k + 1;
}

```

How the time complexity will be changed if $i*i$ replaced with i ?

Ans:

Outer Loop: The outer loop doubles i every time, so it runs **$O(\log n)$** times.

Inner Loop: The inner loop runs i^2 times for each iteration of the outer loop.

Total Complexity: The total time complexity is:

$$(O \sum_{i=1}^{\log n} i^2 = O(n^3))$$

Time Complexity: $O(n^3)$

Now, if i^2 is replaced with i :

Outer Loop: The outer loop runs **$O(\log n)$** times.

Inner Loop: The inner loop runs **i** times for each iteration of the outer loop.

Total Complexity: The total time complexity is:

$$(O \sum_{i=1}^{\log n} i = O(n^2))$$

Time Complexity: $O(n^2)$

06. Find the asymptotic bound of the following recurrence by using master method:

$$T(n) = 4T(n/k) + n^2$$

where (i) $k = 4$ and (ii) $k = 2$.

Ans:

(i) $k = 4$

$$T(n) = 4T\left(\frac{n}{2}\right) + n$$

We know the master theorem,

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

Here, $a = 4$, $b = 2$, $f(n) = n$

Now,

$$T(n) = n^{\log_b a} U(n)$$

$$T(n) = n^{\log_2 4} U(n)$$

$$T(n) = n^2 U(n)$$

$$h(n) = \frac{f(n)}{n^{\log_b a}} = \frac{f(n)}{n^{\log_2 4}} = \frac{n^2}{n^3} = \frac{1}{n} = n^{-1}$$

We know,

$$n^r, r < 0 = O(1)$$

Now,

$$T(n) = n^2 U(n) = n^2 * 1 = n^2$$

$$T(n) = O(n^2)$$

(ii) $k = 2$

$$T(n) = 2T\left(\frac{n}{2}\right) + n$$

We know the master theorem,

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

Here, $a = 2$, $b = 2$, $f(n) = n$

Now,

$$T(n) = n^{\log_b a} U(n)$$

$$T(n) = n^{\log_2 2} U(n)$$

$$T(n) = n U(n)$$

$$h(n) = \frac{f(n)}{n^{\log_b a}} = \frac{f(n)}{n^{\log_2 2}} = \frac{n^2}{n^2} = 1$$

$$(\log n)^r, i \geq 0 = \frac{(\log_2 n)^{i+1}}{i+1}$$

$$(\log_2 n)^0 * 1$$

$$\frac{(\log_2 n)^{0+1}}{0+1} = \log_2 n$$

$$T(n) = O(\log_2 n)$$

07. Compare the time complexity of the following two functions. Which one runs faster?

void Chip (int n)

```
{
    for(int i = 1; i <= n; i = i++)
        for(int j = 1; j <= i; j = j*2)
            k = k + 1;
}
```

void Dale (int n)

```
{
    for(int i = 1; i <= n; i = i*2)
        for(int j = 1; j <= i; j = j++)
            k = k + 1;
}
```

Ans:

Function: Chip(int n)

```
for(int i = 1; i <= n; i++) {
    for(int j = 1; j <= i; j = j * 2) {
        k = k + 1;
    }
}
```

Outer Loop:

- Runs from 1 to n → **n iterations**

Inner Loop:

- $j = 1; j \leq i; j = j * 2 \rightarrow \log_2(i)$ times

So total operations:

$$\sum_{i=1}^n \log_2(i) \approx \log_2(n!) \approx n \log_2 n$$

Time Complexity of Chip = $O(n \log n)$

Function: Dale(int n)

```
for(int i = 1; i <= n; i = i * 2) {  
    for(int j = 1; j <= i; j++) {  
        k = k + 1;  
    }  
}
```

Outer Loop:

- $i = i * 2 \rightarrow \log_2(n)$ times

Inner Loop:

- Runs from 1 to $i \rightarrow i$ times

So total operations:

$$\sum_{i=1,2,4,\dots}^n i = 1 + 2 + 4 + 8 + \dots + n \approx 2n - 1$$

Time Complexity of Dale = $O(n)$

Function Time Complexity Faster?

Chip $O(n \log n)$ ✗

Dale $O(n)$ ☒ Faster

08. Let $K = ((X \text{ MOD } 2) + 1) * 2$, where X is the last digit of your ID. Now, find the asymptotic upper bound of the following recurrence by using master method.

$$T(n) = KT(n/2) + n$$

Verify your answer through recursion tree method.

Ans:

$$\begin{aligned} K &= ((3 \text{ MOD } 2) + 1) * 2 \\ &= (1 + 1) * 2 \\ &= 4 \end{aligned}$$

So, the equation is,

$$T(n) = 4T\left(\frac{n}{2}\right) + n$$

We know the master theorem,

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

Here, $a = 4$, $b = 2$, $f(n) = n$

Now,

$$\begin{aligned} T(n) &= n^{\log_b a} U(n) \\ T(n) &= n^{\log_2 4} U(n) \\ T(n) &= n^2 U(n) \end{aligned}$$

$$h(n) = \frac{f(n)}{n^{\log_b a}} = \frac{f(n)}{n^{\log_2 4}} = \frac{n^2}{n^3} = \frac{1}{n} = n^{-1}$$

We know,

$$n^r, r < 0 = O(1)$$

Now,

$$\begin{aligned} T(n) &= n^2 U(n) = n^2 * 1 = n^2 \\ T(n) &= O(n^2) \end{aligned}$$

Recursion Tree Verification:

Now, let's verify using the recursion tree method:

- **Level 0:** n
- **Level 1:** $4 \times (n/2) = 2n$
- **Level 2:** $4^2 \times (n/4) = 4n$
- **Level k:** $4^k \times (n/2^k) = n \times 2^k$

The total work across all levels is:

$$Total\ work = n + 2n + 4n + \dots + n \times 2^k$$

Summing up the geometric series, we get $O(n \cdot 2^k)$, and since the tree depth is $\log_2 n$, the total work is $O(n \cdot \log n)$, confirming the $O(n^2)$ result.

Final Answer:

So, the asymptotic upper bound of the recurrence is:

$$T(n) = O(n^2)$$

09. Show that the asymptotic upper bound of the following function is not $O(n^2)$: $An^3 + Bn^2 + C$. Assume that A, B, and C are any constants.

OR

Show that the asymptotic lower bound of the following function is not $\Omega(n^2)$: $An^3 + Bn^4 + Cn^5$. Assume that A, B, and C are any constants.

Ans:

Suppose,

$$An^3 + Bn^2 + C \text{ is } O(n^2)$$

So, there will exist a constant k and n_0 such that $n \geq n_0$

$$An^3 + Bn^2 + C \leq k$$

$$\text{or, } An + B + C/n^2 \leq k$$

So, now for any value n the dominant part will always be the left side of equation. It totally contradicts our assumption.

So, $An^3 + Bn^2 + C$ is not upper bounded by $O(n^2)$.

Or,

Suppose,

$$An^3 + Bn^4 + Cn^5 \text{ is } \Omega(n^2)$$

So, there will exist a positive constant k and n_0 such that $n \geq n_0$

$$kn^2 \geq An^3 + Bn^4 + Cn^5$$

$$\text{or, } k \geq An + Bn^2 + Cn^3$$

for any value n the function will hold the condition we assumed.

So, $An^3 + Bn^4 + Cn^5$ is bounded by $\Omega(n^2)$.

10. Find the asymptotic upper bound of the following recurrence by using substitution method.

$$T(n) = (2 * (X \text{ MOD } 3) + 2)T(n/4) + n^2$$

Ans:

Let,

$$T(n) = (2 * (X \text{ MOD } 3) + 2)T(n/4) + n^2$$

Id last digit X = 3

So,

$$T(n) = 2T\left(\frac{n}{4}\right) + n^2$$

Replace, $n \rightarrow \frac{n}{4}$

$$T\left(\frac{n}{4}\right) = 2T\left(\frac{n}{16}\right) + \left(\frac{n}{4}\right)^2$$

Putting eqn 2 in 1

$$\begin{aligned} T(n) &= 2\left(2T\left(\frac{n}{16}\right) + \left(\frac{n}{4}\right)^2\right) + n^2 \\ &= 4T\left(\frac{n}{16}\right) + \frac{n^2}{8} + n^2 \end{aligned}$$

Replace, $n \rightarrow \frac{n}{16}$

$$T\left(\frac{n}{16}\right) = 2T\left(\frac{n}{64}\right) + \left(\frac{n}{16}\right)^2$$

Putting eqn 4 in 3

$$\begin{aligned} T(n) &= 4\left(2T\left(\frac{n}{64}\right) + \left(\frac{n}{16}\right)^2\right) + \frac{n^2}{8} + n^2 \\ &= 8T\left(\frac{n}{64}\right) + 4\frac{n^2}{256} + \frac{n^2}{8} + n^2 \\ &= 8T\left(\frac{n}{64}\right) + \frac{n^2}{64} + \frac{n^2}{8} + n^2 \end{aligned}$$

So, General eqn may be,

$$T(n) = 2^i T\left(\frac{n}{4^i}\right) + n^2\left(\frac{1}{2^{i-1}} + \frac{1}{2^{i-2}} + \cdots + 1\right)$$

Let,

$$\frac{n}{4^i} = 1 \Rightarrow n = 4^i \Rightarrow \log_4 n = i$$

$$i = \log_4 n$$

So, new general equation becomes,

$$T(n) = 2^{\log_4 n} T(1) + n^2(\text{sum of series})$$

Now,

$$2^{\log_4 n} = n^{\log_4 2} = n^{1/2} = \sqrt{n}$$

Since, n^2 dominates $\sqrt{n}$ we get,

$$T(n) = O(n^2)$$

Time complexity $O(n^2)$